

TECHNICAL DEEP DIVE

从 HTTP 到下一枚 Token: vLLM 请求如何穿过三个执行边界

基于 vLLM `releases/v0.20.0`, 并限定 `VLLM\_USE\_V2\_MODEL\_RUNNER=0`
的单请求全生命周期解析

基于技术演讲与配套材料整理

配套材料: vLLM_Request_Journey.pptx

2026-09-05

原视频： [一个 Request 的完整旅程](#) · **配套资料：** [配套资料目录](#)

客户端看到的是一次聊天补全：提交消息，等待文本。vLLM 面对的却是三个节奏完全不同的问题 - HTTP 接口按请求组织语义，调度器需要跨请求分配资源，GPU 则希望持续执行动态批次。

因此，一次请求不是沿着同步调用栈“进入模型再原路返回”。它会先在 API Server 中完成渲染、分词和可选的多模态处理，再跨进程进入 Engine Core；Core 以多轮 `step()` 调度 GPU Worker，最终由另一条共享输出链路把 token 还原成文本，并路由回原请求。

本文沿真实因果链解释这套协议，而不是按照演讲时间或 PPT 页码逐页复述。

适读人群与前置知识

本文面向了解 Transformer 推理、异步服务和 GPU 基础，但尚未熟悉 vLLM 内部请求链路的工程师。阅读前最好已经理解：

- token、logits、prefill 与 decode 的基本含义；
- 异步生成器、进程内队列和跨进程通信的区别；
- KV Cache 如何复用自回归推理中的历史注意力状态；
- Python 调用栈和少量 PyTorch 风格伪代码。

阅读目标

读完后，你应该能够：

- 建立 API Server、Engine Core、GPU Worker 三个执行边界的全局模型；
- 理解请求为何要拆成 `EngineInput` 与 `SamplingParams`；
- 追踪 `SchedulerOutput`、`InputBatch`、`ModelRunnerOutput`、`EngineCoreOutputs` 和 `RequestOutput` 的转换；
- 解释 continuous batching 如何由调度决策、KV Cache 与持久化批状态共同支撑；
- 定位模型前向、结构化约束、采样及 CPU/GPU 同步发生的位置；
- 理解共享输出循环如何把批量结果路由给逐请求生成器；
- 判断结构化输出、投机解码、取消、LoRA 和并行拓扑怎样约束主循环；
- 分清架构事实、合理推断与材料没有支持的性能结论。

一、一个 HTTP 请求，为什么不能直接交给模型

HTTP 接口追求逐请求语义：每个客户端提交自己的消息，并独立等待结果。GPU 推理面对的却是另一种节奏：多个请求需要持续调度，模型每轮只推进一部分状态，各请求的完成时间也不相同。

如果把网络连接、请求调度和设备计算锁进同一条调用栈，慢客户端、动态批次与多轮生成便会相互牵制。vLLM 因而把主链路拆成三个执行边界，并用三类并发循环维持它们各自的节奏。

首先看全局图，因为后续所有数据对象和调用关系都可以放回这三个边界中定位。



图 1：单个请求的跨进程主链路。来源：演讲 PPT，第 4 页。

图中各区域的职责如下：

- 蓝色 **API Server** 接收 HTTP 请求，完成解析、渲染、分词、多模态处理和引擎提交；结果返回后，它还负责输出处理、反分词及 SSE 或 JSON 响应组装。
- 绿色 **Engine Core** 是单轮推理的编排中心。`EngineCore.step()` 依次组织 Schedule、Model Execution、Sample 和 State Update；只要还有未完成请求，Core 就继续下一轮。
- 橙色 **GPU Worker** 执行 Core 安排的设备侧工作，包括输入张量准备、模型前向和采样。它不是主循环的自主驱动者。
- 黄色 **ZMQ** 是 API Server 与 Engine Core 之间的跨进程通信层。请求沿 `ROUTER → DEALER` 进入 Core，输出沿 `PUSH → PULL` 返回 API 进程。
- 紫色路径表示输出经过后台处理后回到客户端。

这些箭头只表达逻辑数据流，不代表真实耗时比例。框体大小也不能用于推断吞吐量、队列容量、显存占用或数据规模。

三类循环分别推进什么

异步生成器（`async generator`）是一种可以等待新结果并多次 `yield` 的调用形式。它很适合表达“一个请求逐步得到输出”，却不适合承担全局调度或直接控制 GPU。

循环	上游生产者	消费与产出	所在进程
逐请求 <code>generate()</code>	共享输出处理器向该请求的 <code>RequestOutputCollector</code> 写入结果	从专属 collector 取出 <code>RequestOutput</code> ，逐次 <code>yield</code> ，直到请求结束	API 进程

循环	上游生产者	消费与产出	所在进程
共享 Output Handler	通信层接收 Core 经 ZMQ 发回的输出并写入 <code>outputs_queue</code>	消费 <code>outputs_queue</code> ，调用 <code>process_outputs()</code> ，再分发到各请求 collector	API 进程
<code>run_busy_loop()</code>	请求经 ZMQ 到达 Core 的输入通道	处理控制消息、调用 <code>engine step</code> ，并将本轮结果写入 <code>output_queue</code>	Core 进程

几个名称相似的对象必须分开：

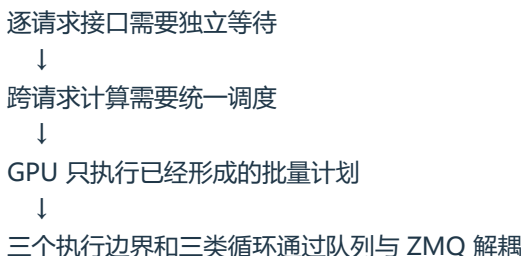
- `output_queue` 位于 Core 进程，用来承接本轮 Core 输出；
- `outputs_queue` 位于 API 进程，承接跨进程返回的数据；
- `RequestOutputCollector` 按 `request_id` 创建，供单个 `generate()` 消费。

因此，`generate()` 既不直接读取 ZMQ，也不直接等待 GPU。

考虑同时到达的请求 A 和 B。API 侧先为它们分别创建 collector，再把请求提交给 Core。Core 连续执行 `step()`，某一轮可能同时推进 A 和 B，并为每个请求产生一个或多个 token。结果经过 Core 的 `output_queue`、ZMQ 和 API 的 `outputs_queue` 后，由共享 Output Handler 按请求分发。此时，A 和 B 对应的 `generate()` 才各自取得结果。

即使 A 的客户端消费较慢，Core 的调度循环也不必和 A 保持在同一调用栈中。

这形成了第一条核心因果链：



还要注意，`step()` 次数不等于输出 token 数。一次 `step()` 可能产生一个或多个 token，一个请求也通常需要多轮推进。

本文所有实现描述都限于 `vLLM releases/v0.20.0`、`VLLM_USE_V2_MODEL_RUNNER=0`，GPU 主链路使用 `v1/worker/gpu_model_runner.py`。这些结论不能直接套用到 V2 Model Runner，也不能代表所有并行部署下的实际进程拓扑。

二、请求进入引擎之前：渲染、分词与多模态分叉

`POST /v1/chat/completions` 接收的 `ChatCompletionRequest` 混合了两类性质不同的信息：

信息类别	典型字段	转换结果
内容输入	<code>model</code> 、 <code>messages</code> ，以及 <code>image</code> 、 <code>audio</code> 、 <code>video</code> 等媒体入口	<code>EngineInput</code> ，即引擎可处理的模型输入
生成配置	<code>temperature</code> 、 <code>max_tokens</code> 、停止条件、 <code>logprobs</code> 、 <code>re sponse_format</code>	<code>SamplingParams</code> ，即生成和采样策略

因此，请求进入引擎前并不只是“把字符串变成 token”。API 层需要把面向协议的请求拆成两条线：

内容与媒体 → `Renderer` → `EngineInput`
 生成配置 → 参数转换 → `SamplingParams`

两者随后共同进入 `AsyncLLM.generate()`。

Serving 层和 Renderer 各自负责什么

下面这张图值得先看，因为它区分了两个容易混淆的层次：谁负责组织预处理，谁真正执行渲染和输入构造。



图 2：渲染流程的四步骨架及其在 Serving 层中的位置。来源：演讲 PPT，第 9 页。

右侧职责链可以分成四层：

- `ServingChat` 接收 `ChatCompletionRequest`，协调健康检查、请求渲染和采样参数转换。
- `ServingRender` 校验 chat template，并编排聊天预处理。
- `Renderer` 执行消息渲染、分词和引擎输入构造。
- `AsyncLLM` 是面向 Engine Core 的客户端，但它与 Engine Core 不在同一个执行边界。

`ServingRender` 与 `Renderer` 因而不是同一层抽象。前者决定采用什么模板以及如何进入预处理；后者把具体请求内容转换成引擎可消费的数据。

`Renderer` 的逻辑过程可归纳为四步：

步骤	输入	主要行为	输出或影响
<code>render_messages()</code>	<code>messages</code>	规范化消息并应用 Hugging Face chat template	<code>prompt</code> 或 <code>prompt_token_ids</code> ，还可能包含 <code>mm_data</code> 、 <code>mm_uuids</code>
<code>tokenize_prompts()</code>	文本提示或已有 token	必要时调用 tokenizer 编码	token ID
<code>apply_prompt_extra_s()</code>	已分词的提示	写入 <code>cache_salt</code> 等附加信息	补充后的 prompt 对象
<code>process_for_engine()</code>	token 与可选多模态数据	构造引擎可消费的输入	纯文本 <code>tokens_input</code> 或多模态输入

这些名称表达逻辑顺序。实际入口包含异步函数，不能据此认为四步全是同步调用。

纯文本请求怎样形成 `tokens_input`

考虑一个最小请求：

```
{
  "model": "example-model",
  "messages": [
    {"role": "user", "content": "解释一下 KV Cache"}
  ],
  "temperature": 0.7,
  "max_tokens": 128
}
```

内容侧的状态演进可以压缩为：

```
messages
→ chat template 渲染后的 prompt
→ prompt_token_ids
→ 写入 cache_salt 等附加字段
→ tokens_input
```

`render_messages()` 先将角色和内容套入模型对应的 chat template。若输出仍是文本，`tokenize_prompts()` 再调用 tokenizer 编码；若上游已有 `prompt_token_ids`，这一阶段会跳过 `tokenizer.encode()`。

`prompt` 与 `prompt_token_ids` 是可选路径，并不保证同时存在。

随后，`apply_prompt_extras()` 加入 `cache_salt` 等附加信息。到 `process_for_engine()` 时，如果没有检测到多模态数据，Renderer 就把 token 包装为后续引擎能够接收的 `tokens_input`。

与此同时，`temperature`、`max_tokens` 等配置不会进入 token 处理链，而是转成单独的 `SamplingParams`。Renderer 在这里“返回”只表示预处理完成，并不表示模型已经开始推理。

多模态请求在哪里真正分叉

多模态（multimodal）请求是包含 image、audio 或 video 等媒体内容的请求。消息解析阶段已经能够携带这些数据，但纯文本与多模态并不是从路由开始就走两套完全独立的流程。

关键分叉发生在 `process_for_engine()`：

没有多模态数据 → `tokens_input`

存在多模态数据 → `_process_multimodal_async()` → `mm_input`

下面这张图需要关注的不是某个具体媒体算子，而是 Hugging Face 与 vLLM 各自负责哪一段转换。



图 3：vLLM `BaseMultiModalProcessor` 与 Hugging Face `ProcessorMixin` 的协作链。来源：演讲 PPT，第 11 页。

从左到右，处理链分为五段：

- 1 原始 image、audio、video 被解析为 `MultiModalDataItems`。
- 2 多模态 UUID 经过解析和处理，与媒体数据一起形成 `ProcessorInputs`。
- 3 `BaseMultiModalProcessor.apply()` 在 vLLM 的编排下调用 Hugging Face `ProcessorMixin`。
- 4 Hugging Face 完成模型特化的媒体预处理，以及 text+media 联合 tokenization，输出 `BatchFeature`。
- 5 vLLM 继续负责缓存、哈希、prompt update 和 placeholder 对齐，并将 `BatchFeature` 转成 `MultiModalKwargsItems`。

两类处理器的职责边界是：

- Hugging Face 处理器掌握具体模型需要的媒体预处理和联合 tokenization；
- vLLM 负责将结果纳入推理服务的数据结构、缓存与提示词对齐体系。

Hugging Face 并不是脱离 vLLM 独立完成整个输入构造，它仍由 vLLM 的多模态链路调用和组织。

最终的 `mm_input` 可用四类代表性字段概括：

- `prompt_token_ids`: 联合处理后的提示 token;
- `mm_kwargs`: 供多模态模型消费的媒体参数;
- `mm_hashes`: 用于多模态缓存相关处理的哈希;
- `mm_placeholders`: 媒体内容与提示词占位位置的对应关系。

这只是字段摘要，不是 `EngineInput` 的完整类型声明。材料也没有给出媒体张量形状、缓存容量、命中率或预处理耗时。

到这里，HTTP 请求完成的只是“可计算化”：内容成为引擎输入，生成选项成为 `SamplingParams`。对象仍位于 API 侧，尚未跨进程，也尚未发生模型前向。

三、先建回程，再发请求：异步提交如何完成结果路由

输入规范化后，系统要解决另一个问题：请求会跨进程执行，结果却必须准确回到发起该请求的 `generate()` 协程。

下面这张图需要从左右两侧分别阅读：左侧是请求提交，右侧是输出回收。两条路径共享 `request_id`，但并不共享一条同步调用栈。



图 4：请求提交与输出回收由两条独立路径完成。来源：演讲 PPT，第 12 页。

Serving 层调用 `AsyncLLM.generate()` 后，内部提交过程依次完成：

- 1 `InputProcessor.process_inputs()` 将 `EngineInput` 转为 `EngineCoreRequest`。
- 2 `OutputProcessor.add_request()` 按 `request_id` 创建 `RequestState` 和逐请求的 `RequestOutputCollector`。
- 3 `EngineCoreClient.add_request_async()` 才通过 ZMQ ROUTER 把请求发送至 Engine Core。

四类关键数据对象的角色如下：

数据对象	所在阶段	主要角色	代表性内容
EngineInput	API 侧预处理之后	汇集已经渲染、分词的引擎输入	token、多模态信息、embeddings
EngineCoreRequest	跨进程提交之前	Engine Core 接收的请求表示	prompt、SamplingParams、多模态特征
EngineCoreOutput	Engine Core 返回时	携带新增的生成结果	token IDs、结束原因、logprobs
RequestOutput	API 侧处理之后	面向 Serving 层的逐请求结果	text、token IDs、logprobs、finished

这些字段同样只是数据流摘要。

为什么 collector 要先创建

`OutputProcessor.add_request()` 会为每个请求建立：

- `RequestState`：保存 API 侧继续处理输出所需的逐请求状态；
- `RequestOutputCollector`：供相应 `generate()` 等待和消费结果的收集器。

最小状态演进如下：

```
注册 request_id=r1
r1 → RequestState(r1)
r1 → collector(r1)
```

```
发送 ADD(r1)
→ Engine Core
```

```
Core 返回 EngineCoreOutput(request_id=r1)
→ 共享 Output Handler
→ process_outputs()
→ collector(r1)
→ generate(r1) 被唤醒
→ yield RequestOutput
```

从控制流可以确认，collector 的注册发生在 ZMQ 发送之前。因此，在 Core 开始产生输出前，本地已经存在确定的路由目标。

把这个设计称为“避免丢失输出”属于基于控制流的合理推断，而不是材料提供的故障统计。材料没有给出乱序实验、队列容量或相关性能数据。

如果同时存在 `r1` 与 `r2`，它们也不会各自读取一条 ZMQ 连接。独立的 Output Handler 统一接收 `EngineCoreOutputs`，处理后再按 `request_id` 写入对应 collector。新增并发请求时，增加的是逐请求状态和 collector，而不是一组互相争抢消息的 ZMQ 消费者。

请求进入 Core 后发生什么

`EngineCoreProc.run_busy_loop()` 先处理输入通道中的控制消息：

- **ADD**：加入新请求；
- **ABORT**：取消已有请求；
- **UTILITY**：材料未继续展开的辅助操作。

处理完消息后，它才调用 `_process_engine_step()`。其中，**ADD** 最终通过 `Scheduler.add_request()` 进入 `waiting`。

进入 `waiting` 只代表请求取得参与调度的资格，不表示它一定会进入当前批次。是否获选还要看 token 预算、序列容量和 KV Cache 资源。

边界上应始终区分：

- 请求跨进程：ROUTER → DEALER
- 输出跨进程：PUSH → PULL
- API 进程内：outputs_queue
- Core 进程内：output_queue
- 逐请求交付：RequestOutputCollector

四、Scheduler 如何把等待队列变成本轮 GPU 批次

`EngineCore.step()` 会反复推进活跃请求，但每一轮都不会把队列中的所有请求直接交给 GPU。它首先调用 `Scheduler.schedule()`，回答三个相互约束的问题：

- 1 本轮选择哪些请求；
- 2 每个请求计算多少 token；
- 3 KV Cache 是否足以支撑这些计算。

一次 `step()` 可以压缩为四个阶段：

阶段	主要动作	产物或状态变化
Schedule	选择请求、分配 token 配额和 KV Cache blocks	<code>SchedulerOutput</code>
Model Execution	准备输入并执行模型前向	暂存 logits
Sample	应用约束并从 logits 采样	<code>ModelRunnerOutput</code>
State Update	追加 token、检查停止条件、更新状态	完成时释放 KV Cache

调度器只负责构造“本轮执行计划”，并不直接执行模型计算。

RUNNING 与 WAITING 如何汇入同一批次

下面这张图的重点是左右关系：左侧展示两类请求如何接受资源检查，右侧展示这些决策如何固化为 `SchedulerOutput`。



图 5: `Scheduler.schedule()` 的两条请求路径及 `SchedulerOutput` 代表性字段。来源：演讲 PPT，第 16 页。

RUNNING 表示请求已经进入执行集合，并不意味着它只能处在 decode 阶段。

- **Prefill** 是处理提示词 token 的计算阶段。
- **Decode** 是利用已有上下文继续生成新 token 的阶段。

对 **RUNNING** 请求，调度器需要：

- 1 计算本轮可调度的 token 数；
- 2 对长 prefill 做切分；
- 3 为即将计算的 token 分配 KV Cache blocks；
- 4 在可用显存块不足时处理抢占。

KV Cache blocks 是承载注意力键值缓存的分块资源。材料没有提供 block 大小、真实地址布局、`long_prefill_token_threshold` 数值、抢占优先级或恢复成本。

WAITING 请求则会经历：

- 1 从等待队列取出候选；
- 2 检查 prefix cache；
- 3 检查序列容量；
- 4 分配 KV Cache blocks；
- 5 满足条件后转入 **running**。

所以，从 **waiting** 进入 **running** 不是简单的先进先出操作。prefix cache、序列容量和 KV Cache 可用块都会改变本轮选择。

SchedulerOutput 怎样描述执行计划

字段	本轮含义
num_scheduled_tokens	从请求 ID 映射到 token 数，表示各请求本轮的计算配额
scheduled_new_reqs	本轮新进入执行集合的请求
scheduled_cached_reqs	已在执行集合中并继续参与本轮计算的请求
block_ids	请求与 KV Cache blocks 之间的映射
finished_req_ids	已完成、需要清理状态的请求标识

后续组件不必重新扫描等待队列，也不必自行判断每个请求该计算多少 token；它们只需执行 Scheduler 已经确定的请求分类、配额和块映射。

这些仍是代表性字段，并非完整类型定义。

三个请求的最小调度例子

设当前有三个请求：

请求	调度前状态	当前阶段	本轮结果
A	RUNNING	decode	获得配额并继续执行
B	WAITING	prefill	通过缓存、容量和 KV 检查，转入 running
C	WAITING	prefill	因 KV Cache blocks 不足，继续等待

本轮 SchedulerOutput 同时包含 A 和 B：

- A 属于已经在执行集合中的请求；
- B 属于本轮新加入的请求；
- 两者都在 num_scheduled_tokens 中获得配额；
- block_ids 记录它们本轮使用的 KV Cache blocks；
- C 没有满足资源条件，因此不进入本轮模型执行。

同一个批次由此可以同时包含 A 的 decode 和 B 的 prefill。

Continuous batching（连续批处理） 指不同请求可以在不同生命周期阶段共同组成当前批次。GPU 不必等待整批请求全部结束 prefill 后，再一起进入 decode。

材料没有给出全局 token 预算、具体配额、批大小或 KV block 数量，因此这里能确认的是结构机制，而不是吞吐提升比例。

KV Cache 如何形成跨轮闭环



若 Phase 1 中资源不足，调度器可能抢占请求，使其回到 `waiting`；请求在 Phase 4 完成后释放 KV Cache，又会改变下一轮可用容量。

因此，continuous batching 并不只是“把请求拼在一起”。它依赖调度决策、KV Cache 生命周期和跨轮持久状态共同成立。

五、从 SchedulerOutput 到 logits: GPUModelRunner 的执行现场

调度器只描述“这一轮算什么”。真正执行前，还要把离散请求、token 配额和 KV block 映射转换成 GPU 能够消费的批量张量。

这里先看三层职责图，因为 Worker、GPUModelRunner 与 Model 经常被笼统地称为“模型执行层”，但它们管理的状态完全不同。



图 6：模型执行阶段的三层组件及调用关系。来源：演讲 PPT，第 17 页。

组件	持有或管理的对象	核心职责	不负责什么
Worker	rank、设备、分布式通信、GPU 内存、ModelRunner	建立设备和并行执行环境	不直接组织请求批次

组件	持有或管理的对象	核心职责	不负责什么
GPUModelRunner	持久化 <code>InputBatch</code> 、执行状态	更新批状态、构造输入、处理多模态数据、调用前向并衔接采样	不实现 Transformer 的逐层计算
Model	网络层和模型参数	完成一次模型前向	不管理调度、请求生命周期和采样

“Model 只负责一次前向”描述的是职责边界，不表示一个请求只执行一次前向。请求进入 decode 后通常跨越多个 `step()`；每轮都可能重新形成批次并调用一次 Model。

InputBatch 为什么要跨轮保留

`InputBatch` 汇总当前活跃请求对应的 GPU 状态，并跨迭代持久化。其作用可以通过两个请求的状态演进观察：

时刻	请求 A	请求 B	InputBatch 的变化
第 k 轮开始	正在 decode	尚未进入执行	保留 A
第 k 轮调度后	继续 decode	新加入并执行 prefill	在 A 的基础上加入 B
第 k+1 轮	未结束则保留，结束则移除	转入 decode 或继续 prefill	清理完成项，保留其余活跃项

新请求加入时不必丢弃已有 decode 请求；旧请求结束后也不会继续占据活跃批状态。`InputBatch` 的“保留、加入、移除”让批次能够随每轮调度变化。

execute_model() 如何落实调度结果

下面这张图要沿调用树阅读：左侧是持久状态和输入张量的更新，右侧是多模态编码与两条前向分支。



图 7：execute_model() 从状态更新到 logits 暂存的调用树。来源：演讲 PPT，第 18 页。

主链路可以概括为：

```
SchedulerOutput
→ _update_states()
→ _prepare_inputs()
→ _preprocess()
→ Model forward
→ execute_model_state
```

第一步, `_update_states()` 把调度结果落实到 GPUModelRunner 的持久状态中。它会:

- 清理已完成请求的 cached state;
- 更新 KV block table;
- 更新采样元数据;
- 更新 LoRA 状态;
- 处理新建、恢复或继续运行的请求;
- 清理相应的 encoder cache 状态。

因此, `SchedulerOutput` 不会直接传给 Model。它先改变 GPUModelRunner 对当前活跃批次的认识。

第二步, `_prepare_inputs()` 构造真正参与本轮计算的输入。材料给出的批内布局是:

decode 在前, prefill 在后

同时还会构造 `idx_mapping`、`query_start_loc` 等辅助张量。这个排列顺序不表示 prefill 与 decode 必须分别执行两次前向; 它们仍可以组成一次模型调用。

两类输入的准备方式不同:

- prefill 使用 Triton kernel 填充 `input_ids`;
- decode 合并上一 token 与 draft tokens。

后者对应投机解码跨轮携带的候选 token。材料没有给出 Triton kernel 配置、线程组织、张量形状或相关性能数据。

回到双请求例子: 第 k 轮中, A 的 decode 输入位于批次前部, B 的 prefill 输入位于后部; 辅助索引张量负责描述各请求的查询边界。生命周期不同的请求由此可以共享一次批量前向, 而不会失去各自的序列范围。

多模态 embedding 如何进入主模型

若请求包含多模态内容, GPUModelRunner 会按 modality 分组, 并调用:

```
model.embed_multimodal(**mm_kwargs)
```

编码结果以 `mm_hash` 为键写入 `encoder_cache`。随后, 系统汇集所需多模态 embedding, 并将它们合并为 `inputs_embeds`, 再交给语言模型主干。

这里存在两种不能混淆的缓存:

- `encoder_cache` 保存多模态编码结果;
- KV Cache 保存自回归注意力计算中的历史键值状态。

材料只展示了

`encoder_cache[mm_hash]`

的索引关系, 没有说明哈希算法、碰撞处理、缓存容量或淘汰规则。

两条前向路径与一个 logits 落点

输入准备完成后, 模型前向会在两条路径中选择其一:

- CUDA Graph 路径调用 `graph.replay()`;
- eager 路径调用 `model(**model_inputs)`。

它们是替代分支, 不是同一轮中依次执行的两个阶段。

前向产生的 logits 不作为 `execute_model()` 的直接返回值, 而是暂存在 `execute_model_state` 中; 该路径上的 `execute_model()` 返回 `None`。

这是一种阶段化接口语义:

`execute_model()` 返回 `None`
≠ 没有计算结果

实际含义:

logits 留在执行状态中, 等待 `sample_tokens()` 消费

模型执行以非阻塞方式提交时, CPU 可以同时准备结构化输出所需的
bitmask; 采样前再通过 `future.result()` 等待
完成。但材料中的图没有时间刻度, 也没有提供重叠时长, 不能据此计算性能收益。

grammar
GPU

六、从 logits 到状态迁移: 采样如何闭合一次 `step()`

模型前向结束时得到的是 logits, 即词表中各候选 token 的分数。一次 `step()` 要闭合, 还需完成三件事:

- 1 根据请求配置和语法约束修改候选空间;
- 2 从处理后的 logits 中选出 token;
- 3 把 token 写回请求状态, 并决定是否进入下一轮。

logits 怎样变成 token

下面这张图需要按箭头顺序阅读, 同时注意两个分支关系: grammar 和 logits processors 先修改分数, 而随机采样与 greedy 是替代路径。



图 8: 采样、同步与输出形成路径。来源: 演讲 PPT, 第 19 页。

链路可以压缩为:

```
execute_model_state.logits
→ grammar bitmask
→ logits processors
→ top-k/top-p 或 greedy/argmax
→ _bookkeeping_sync()
→ ModelRunnerOutput
```

`sample_tokens()` 首先取出 GPU 上暂存的 logits。若请求带有结构化输出约束, 系统会应用 **grammar bitmask**: 由当前语法状态产生一个 token 可用性掩码, 并把非法 token 的 logit 置为负无穷。

以三个抽象候选为例:

token	原始 logit	grammar 是否允许	约束后 logit
A	4.2	是	4.2
B	5.1	否	$-\infty$
C	3.7	是	3.7

B 原本分数最高, 但被语法禁止后不再可能被采中。约束不是对最终文本做事后校验, 而是在采样前改变候选集合。

之后, `apply_logits_processors()` 继续处理分数。材料列出的处理类别包括:

- `logit_bias`;
- `allowed_token_ids`;
- `frequency`、`presence` 和 `repetition penalties`;
- `temperature`;

- min-p;
- top-k;
- top-p。

并非每个请求都会启用全部处理器。

处理完成后进入采样分支：

- 随机采样可以使用 FlashInfer 的 top-k/top-p GPU kernel;
- 确定性生成走 greedy/argmax。

两者是替代关系。

`_bookkeeping_sync()` 随后把 GPU tensor 转成 CPU 侧的 `list[int]`，形成包含 sampled IDs 和可选 `logprobs` 的 `ModelRunnerOutput`。材料把它称为 `sample_tokens()` 这段链路的唯一同步点；这一结论不能扩展成“整个请求生命周期只有一次 CPU/GPU 同步”。

Scheduler 如何推进请求状态

取得 sampled IDs 后，`Scheduler.update_from_output()` 继续完成：

- 1 把采样结果追加到 `request.output_token_ids`;
- 2 检查 EOS、`max_tokens` 和 `stop_token_ids`;
- 3 对结构化输出调用 `grammar.accept_tokens()`;
- 4 若请求结束，释放 KV Cache，并生成 `EngineCoreOutput`。

对于结构化输出，bitmask 与 `accept_tokens()` 分别承担两个方向的状态转换：

语法状态 G0
→ 计算 mask(G0)
→ 屏蔽非法 token
→ 采中 token A
→ `grammar.accept_tokens(A)`
→ 语法状态 G1

当前语法状态决定这一轮可以选择什么；采样结果又决定下一轮处于什么状态。

停止逻辑则分布在两个执行边界：

- Engine Core 处理 EOS、`max_tokens` 和 `stop_token_ids`;
- API 侧在反分词后检查 stop strings，命中时才可能向 Core 发送 abort。

因此，Core 完成 token 级停止检查，并不代表所有字符串级停止条件也已处理。

结构化输出与投机解码为何不是局部开关

下面这张图需要对照左右两条路径：结构化输出跨越请求参数、约束和语法状态；投机解码则跨越相邻两轮。



图 9：结构化输出和 Speculative Decoding 在流水线中的落点。来源：演讲 PPT，第 25 页。

结构化输出从 `response_format` 开始：

- `response_format`
 - `SamplingParams`
 - `StructuredOutputRequest`
 - grammar 异步初始化
 - grammar bitmask 约束 logits
 - `grammar.accept_tokens()` 推进状态

它不是采样器上的单个布尔开关，而是一条从 API 请求语义延伸到逐 token 状态迁移的链路。

Speculative Decoding（投机解码） 则具有跨迭代依赖：

- 上一轮：提出 draft tokens
 - ↓
- 下一轮：将 draft 计入 token 预算
 - ↓
- 一次 forward 验证候选
 - ↓
- 接受或拒绝，并修正计算状态

关键在于“上一轮到下一轮”的箭头。draft 产生后不能在当前采样函数内独立完成全部语义；下一轮要把它计入预算，模型前向负责验证，状态更新再决定接受或拒绝，并相应调整 `num_computed_tokens`。

tokens
Scheduler

因此，把投机解码概括成“更快的 draft、下一轮预算、验证结果和计算进度修正。

sampler”会遗漏决定正确性的状态：待验证

材料没有给出 draft 数量、接受率、回退公式或可复现的加速数据。这里能够确认的只有跨轮机制。

七、Token 回到文本：共享输出循环如何服务逐请求响应

得到 token ID 后，客户端仍然看不到文本。Core 的批量结果还需要跨进程返回，完成增量反分词、字符串停止判断和逐请求路由。

返回路径如下：

```
EngineCoreOutputs
→ Core output_queue
→ ZMQ PUSH / msgpack
→ ZMQ PULL
→ API outputs_queue
→ process_outputs()
→ RequestOutputCollector
→ AsyncLLM.generate()
```

Engine Core 先把结果写入本进程的 output_queue，再经 ZMQ 发往 API 进程的 outputs_queue。共享 Output Handler 统一消费这些输出；处理完成后，再根据 request_id 投递到对应 collector。

下面这张图要从左向右阅读：左侧描述 token 如何成为 RequestOutput，右侧展示增量反分词的默认路径与回退路径。



图 10：输出从 token 更新为逐请求结果，再被封装为流式响应；右侧给出反分词的默认与回退路径。来源：演讲 PPT，第 22 页。

Stage 10：从新增 token 到 RequestOutput

process_outputs() 承担四项关键工作：

环节	作用
Detokenizer.update()	对新增 token 做增量反分词
LogprobsProcessor.update()	请求需要 logprobs 时更新相关输出

环节	作用
stop strings 检查	获得文本后判断字符串停止条件
<code>RequestState.make_request_output()</code>	汇总文本、完成状态和可选信息，构造 <code>RequestOutput</code>

增量反分词（incremental detokenization）不会在每轮重新解码完整 token 历史。默认路径使用 Rust `DecodeStream` 对应的 `FastIncrementalDetokenizer`，无法使用时回退到 Python `SlowIncrementalDetokenizer`。

假设三轮累计 token 为：

轮次	累计序列	本轮新增
1	<code>[t_1]</code>	<code>t_1</code>
2	<code>[t_1, t_2]</code>	<code>t_2</code>
3	<code>[t_1, t_2, t_3]</code>	<code>t_3</code>

第三轮基于已有解码状态处理新增的 `t_3`，而不是从 `t_1` 重新开始。材料没有提供快慢两条路径的性能差值，也没有展开 `stream_interval` 的具体缓冲规则。

字符串停止条件只能在获得文本后判断。因此：

`stop_token_ids` → Core 侧按 token 检查
`stop strings` → API 侧反分词后检查

当 `stop strings` 命中且 Core 中的请求仍需终止时，API 侧才可能发送 abort；不能写成所有完成请求都会反向取消。

处理完成后，`RequestOutput` 被放入对应 collector。`AsyncLLM.generate()` 只消费自己的 collector：未完成时继续 `yield`，完成后退出。

Stage 11：流式与非流式在哪里分开

流式路径把每次得到的 `RequestOutput` 转为 `ChatCompletionStreamResponse`，再编码成服务器发送事件（Server-Sent Events，SSE）中的 data 帧。

响应增量可以包含 `content`、`reasoning_content` 和可选 `usage`。下面只示意外形，不代表完整字段集合：

`data: {"delta":{"content":"系统"}}`

`data: {"delta":{"content":"完成"}}`

`data: [DONE]`

`[DONE]` 表示 SSE 流结束。

非流式请求不会绕开前面的推理、跨进程返回和输出处理链路。它同样消费的结果，只是在最后一次输出到达后组装完整 JSON。

`generate()`

所以，流式与非流式共享同一生成主链，主要区别位于最终响应组装边界：

同一 RequestOutput 流
├─ 流式：逐次封装为 SSE
└─ 非流式：等待最终输出后组装 JSON

材料没有展开错误帧、断连行为、完整分块格式、队列容量或通信耗时。

八、主链路之外的约束：取消、LoRA、多卡与证据边界

主循环并不只受 token 生成驱动。取消会改变请求退出时间，LoRA 会改变批次准入和图特化条件，并行拓扑则会改变 Worker 协作方式和跨 EngineCore 路由。

Abort 为什么不能打断当前 forward

下面这张图要沿 API、Core、Worker 三层观察取消消息。右侧的 LoRA 路径则说明，一个请求参数如何一直影响到 Scheduler 与 GPU 执行条件。



图 11：取消请求与 LoRA 对主循环的约束。来源：演讲 PPT，第 26 页。

取消路径如下：

- 1 API 层的 `generate()` 捕获 `CanceledError`;
- 2 API 调用 `abort()`，通过 ZMQ 发送 `ABORT`;
- 3 Engine Core 在两次 `step()` 之间处理取消;
- 4 Worker 到下一次 `_update_states()` 才把请求移出持久化 `InputBatch`，并清理对应 encoder cache。

Abort 因而是异步控制消息，不是抢占式 GPU 中断。

若请求 R 在本轮 forward 中途被取消，当前设备执行不会立即停止。若消息错过本轮状态处理点，R 最坏可能再执行一轮，随后才从批次移除。这是流程上界，不代表每次取消都会多执行一轮，也不能换算成固定时间。

LoRA 如何进入调度与执行条件

LoRA (Low-Rank Adaptation, 低秩适配) adapter 从 `ChatCompletionRequest` 进入系统:

请求指定 adapter

- Scheduler 跟踪 active LoRAs
- 受 `max_loras` 约束
- Worker 将相应权重加载到 GPU
- CUDA Graph 按 `num_active_loras` 特化

这里能够确认的是:

- active LoRAs 会参与 Scheduler 的批次约束;
- 同时活跃的 adapter 数量受 `max_loras` 限制;
- Worker 负责设备侧权重加载;
- CUDA Graph 的特化条件包含活跃 LoRA 数量。

材料没有给出 `max_loras` 数值、加载耗时、显存成本或性能变化。按活跃数量特化也不等于每个 adapter 名称必然对应一张独立图。

TP、PP 与 DP 改变了什么

下面这张图要分成两类问题阅读: TP/PP 关注一组 Worker 如何协同完成同一次 forward, DP 则引入多个 EngineCore 以及请求路由。



图 12: 并行拓扑、进程数量与请求路由状态。来源: 演讲 PPT, 第 27 页。

张量并行 (Tensor Parallelism, TP) 与流水线并行 (Pipeline Parallelism, PP) 下, 一个 EngineCore 管理一组协同完成同一模型 forward 的 Worker。对前端请求而言, 组内拆分是透明的, 路由目标仍是这个 EngineCore。

数据并行 (Data Parallelism, DP) 则存在多个 EngineCore, 每个 EngineCore 管理一组 Worker。EngineCoreClient 或外部负载均衡需要先选择目标 DP rank。

材料给出的 GPU Worker 数关系为:

$$N = DP \times PP \times TP$$

其中 N 表示 GPU Worker 数。例如 DP=2、PP=1、TP=4 时，公式得到 8 个 Worker。

这不能继续推导为“必需 8 张物理 GPU”，因为材料没有证明 Worker 与物理 GPU 始终一一对应。API Server 数量记为 A，但材料也没有给出 A 与 DP 的固定关系。

当 `DP>1` 时，还存在一个 DP Coordinator，用于协调队列计数和 wave 进度，而不是直接执行模型前向。

DP 路由需要保存两类状态：

- `EngineCoreRequest.client_index`：标识输出应返回哪个 frontend；
- `request_id → engine` 路由表：记录请求实际由哪个 EngineCore 持有。

前者用于输出回送，后者用于把 abort 发往正确的 DP rank。它们解决的不是模型计算问题，而是多 EngineCore 环境中的请求归属问题。

证据边界与源码阅读顺序

本文能够确认的是机制及其潜在影响：

- Abort 可能让已取消请求至多多执行一轮；
- LoRA 增加批次准入、GPU 权重加载和 CUDA Graph 特化条件；
- TP/PP 改变协同 forward 的 Worker 组织；
- DP 增加 EngineCore 选择、输出回送和取消路由状态。

但材料没有提供硬件、模型、batch、并行度、负载、端到端延迟、吞吐量、显存占用或基准方法，因此不能给出确定的加速比或性能收益。

问答中关于投机解码“接近三倍”和“约 2.7”的说法缺少模型、硬件、候选数、接受率和测试方法，只能视为低置信度口头信息，不进入本文性能结论。

如果要沿源码复核，可按以下顺序阅读：

- 1 API Server、Engine Core 与 GPU Worker 的进程边界及 ZMQ 两端；
- 2 Chat Completions 请求解析与 Renderer；
- 3 `EngineCore.step()`；
- 4 Scheduler 的 `schedule()` 与 `update_from_output()`；
- 5 `GPUModelRunner.execute_model()` 与 `sample_tokens()`；
- 6 `OutputProcessor`、Detokenizer 和 SSE generator。

源码行号会随版本变化。函数名和数据流才是主要阅读锚点，演讲中的行号不能当作稳定 API。

结论：一次聊天生成是一套跨轮协议

- 一次聊天请求不是一条同步函数调用，而是由 API Server、Engine Core、GPU Worker 三个执行边界，以及共享通道和逐请求状态共同维持的多轮协议。

- API Server 先完成 chat template 渲染、分词和可选多模态处理，再分别形成引擎输入与 `SamplingParams`。这一步决定后续调度和执行能够看到哪些信息。
- `OutputProcessor` 在发送请求前建立 `RequestState` 和 `RequestOutputCollector`。共享接收器据此按 `request_id` 路由结果，而 `AsyncLLM.generate()` 只消费自己的 collector。
- `EngineCore.step()` 通过 Schedule、Model Execution、Sample、State Update 闭合一轮。`SchedulerOutput` 与持久化 `InputBatch` 使 prefill 和 decode 可以混合进入当前批次。
- `execute_model()` 返回 `None` 不表示没有结果。logits 被保存在 `execute_model_state` 中，后续采样完成后才在 `_bookkeeping_sync()` 所描述的同步点形成 CPU 侧输出。
- 结构化输出通过 grammar 状态约束每一轮候选 token；投机解码则把 draft tokens 带入下一轮预算、前向验证和状态修正，因此具有明确的跨迭代依赖。
- 输出由共享 Output Handler 统一接收、增量反分词并检查 stop strings，再投递到逐请求 collector。流式 SSE 和非流式 JSON 复用同一推理与输出主链。
- 取消、LoRA 与 DP/TP/PP 并非孤立的外围功能：它们分别通过跨轮状态清理、批次准入、设备权重、图特化和请求路由约束主循环。

明确局限

本文所有实现结论仅适用于材料所分析的 `vLLM` [releases/v0.20.0](#)，并限定 `VLLM_USE_V2_MODEL_RUNNER=0`。不能把文中的函数职责和执行关系直接推广到 `V2` `Model Runner` 或其他版本。

材料中的阶段、框体和箭头只表示逻辑关系，不带时间比例。现有证据也没有提供足够的硬件、模型、批大小、序列长度、并行度和负载条件，因此本文不对吞吐、延迟、显存占用、CUDA Graph、FlashInfer、结构化输出或投机解码给出量化性能承诺。